UNIVERSIDAD AUTONOMA DE SINALOA
FACULTAD DE INGENIERIA MOCHIS

Licenciatura en Ingenieria de Software

Computo en la Nube - Maestra: Elizabeth Gaxiola Carrillo

PRACTICA 07

Base de datos no relacional y operaciones desde codigo

Equipo: Los Rojos

Integrante	Matricula
Flores Guevara Angel Gabriel	
Lopez Payan Kevin Ricardo	

Los Mochis, Sinaloa, Mexico - 15 de octubre de 2026

1. Introduccion

Una base de datos no relacional no es una base relacional sin reglas: es un modelo distinto en el que el diseno comienza por las consultas, no por las entidades. En esta practica se modelo la tabla `movimientos_inventario` en **Amazon DynamoDB** con capacidad bajo demanda, se implementaron las cuatro operaciones basicas desde codigo, y se comparo el enfoque con el relacional de la practica anterior.

Recursos desplegados

Recurso	Valor
Tabla DynamoDB	movimientos_inventario
Clave de particion (PK)	producto_id (String)
Clave de ordenamiento (SK)	fecha (String) - formato 'ISO-8601#movimiento_id'
Ejemplo de SK	2026-10-14T09:15:00Z#MOV-8810
GSI	GSI_fecha PK=fecha_dia (YYYY-MM-DD) / SK=fecha
Modo de capacidad	PAY_PER_REQUEST (bajo demanda)
ARN	arn:aws:dynamodb:us-east-1:935710521154:table/movimientos_inventario
Region	us-east-1
Etiquetas	Equipo=LosRojos, Proyecto=InventarioNube, Practica=P07

2. Patron de acceso - Diseno antes de la tabla

El error mas comun al disenar una tabla DynamoDB es crear la estructura y despues intentar adaptarla a las consultas. El orden correcto es el inverso: primero se identifican las consultas frecuentes y despues se deriva la clave.

Las tres consultas mas frecuentes del sistema

ID	Consulta	Frecuencia
Q1	Dado producto_id, obtener todos sus movimientos ordenados por fecha descendente	Cada registro de entrada/salida en caja y bodega
Q2	Dado producto_id, obtener el movimiento de salida mas reciente (evitar pedidos duplicados)	Cada vez que se registra una salida
Q3	Dado un rango de fechas, obtener todos los movimientos (corte diario / reporte de gastos)	Periodica - modulo de Reportes y corte diario

Eleccion de claves

Clave	Atributo	Tipo	Justificacion
Particion (PK)	producto_id	String	Q1 y Q2 siempre filtran por producto_id. DynamoDB localiza la particion en O(1).
Ordenamiento (SK)	fecha	String	Formato 'ISO-8601#movimiento_id' (ej: '2026-10-14T09:15:00Z#MOV-8810'). El prefijo ISO garantiza orden cronologico; el sufijo #movimiento_id evita que dos movimientos del mismo producto en el mismo segundo se sobrescriban (serian el mismo item sin el sufijo).

GSI para Q3 - Rango de fechas sin Scan

Q3 no puede resolverse con la clave principal porque cubre **todos los productos** en un rango de tiempo: no hay un producto_id fijo que filtrar. Sin un indice, la unica opcion es un Scan completo.

Nombre	PK del GSI	SK del GSI	Proyeccion	Resuelve
GSI_fecha	fecha_dia (YYYY-MM-DD)	fecha (ISO#movimiento_id)	ALL	Q3: corte diario / rango de fechas sin Scan

El atributo `fecha_dia` (ej: '2026-10-14') se escribe junto con el item al insertarlo. Para un rango de varios dias se itera: fecha_dia='2026-10-14', fecha_dia='2026-10-15', etc.

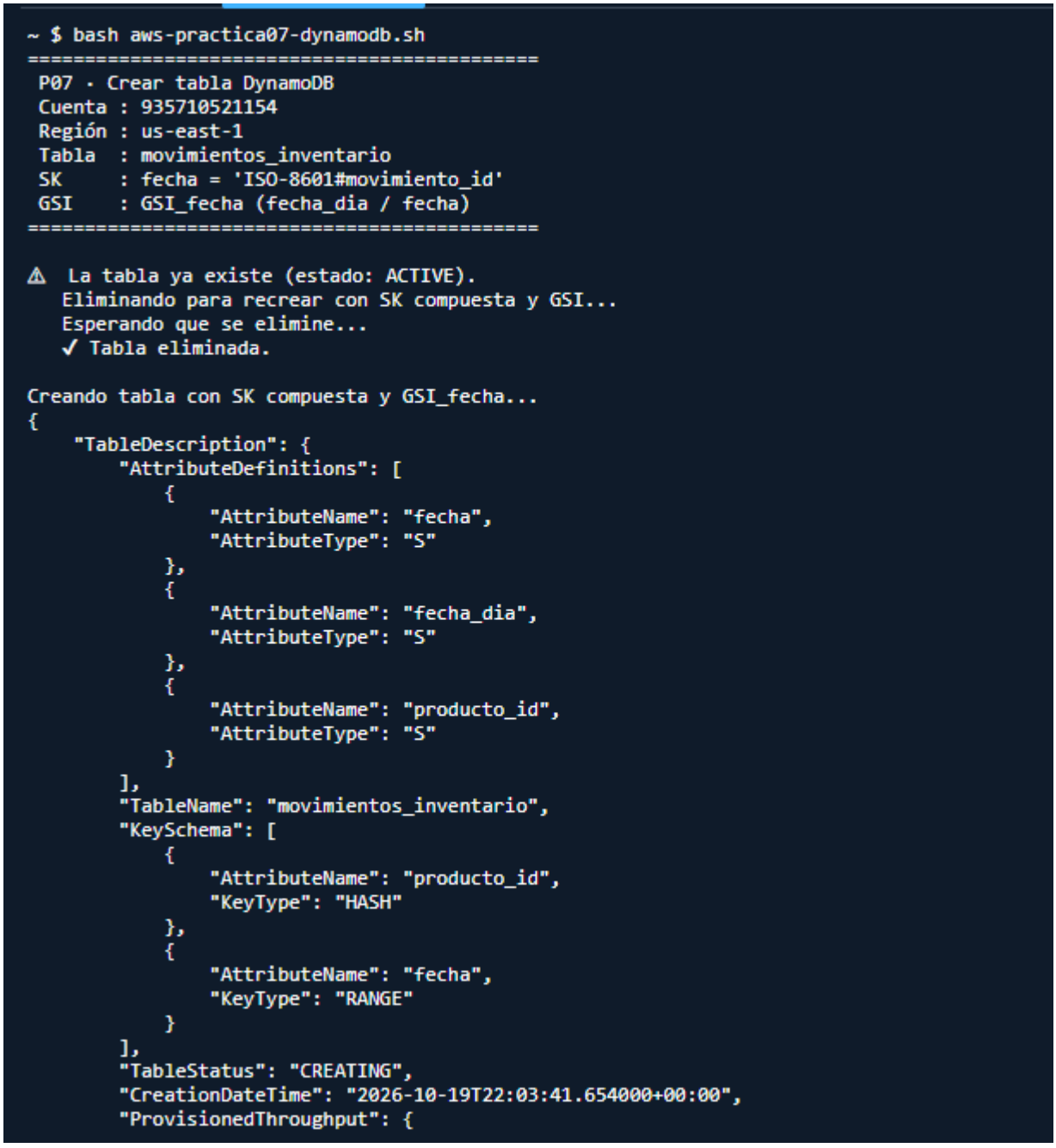
3. Creacion de la tabla

Se ejecuto `aws-practica07-dynamodb.sh` en AWS CloudShell con el rol de servicio. La tabla se creo con **capacidad bajo demanda (PAY_PER_REQUEST)**: AWS aprovisiona la capacidad automaticamente y cobra por operacion, no por hora. Una tabla en reposo no genera ningun costo.

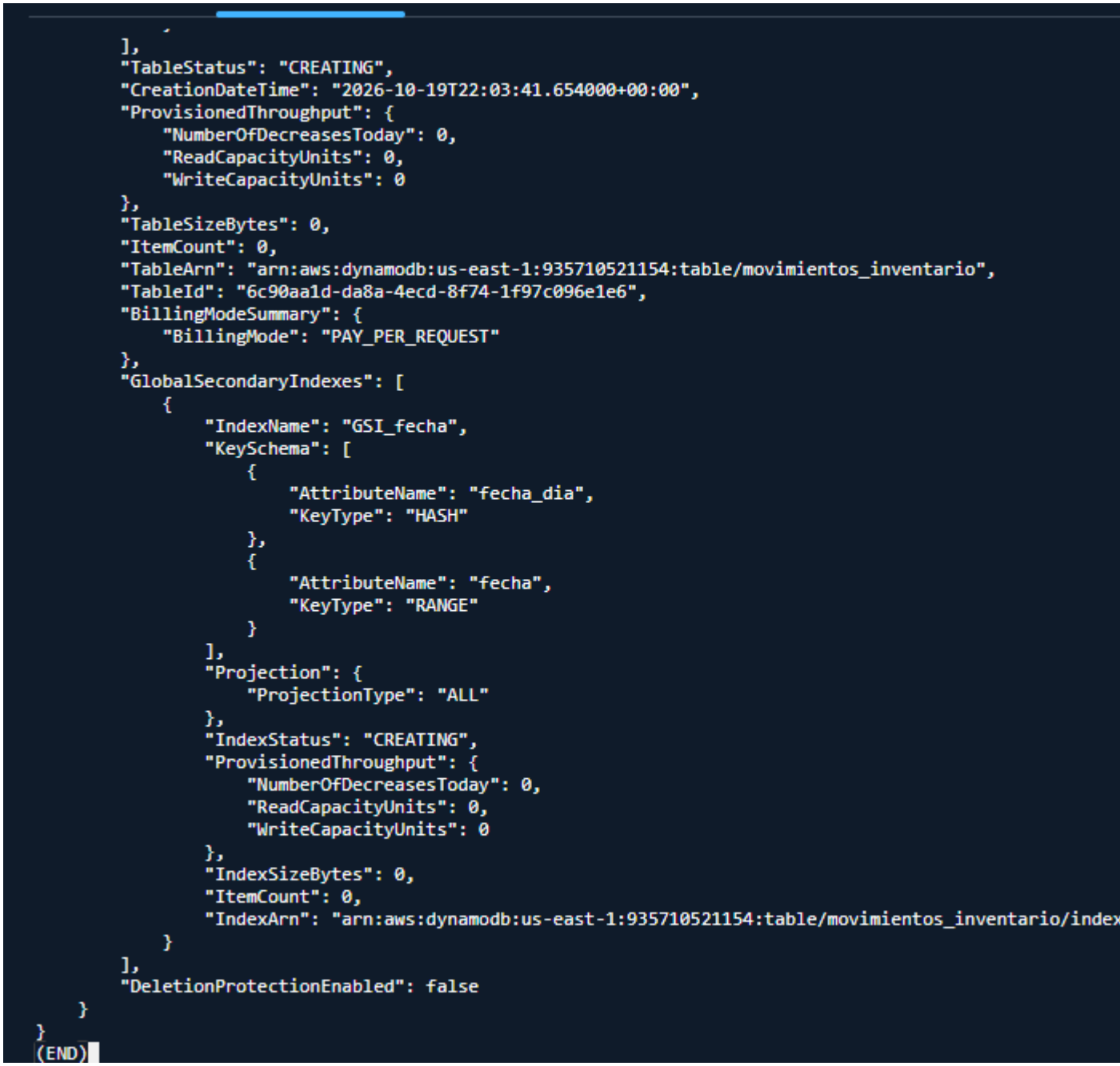
PAY_PER_REQUEST vs. capacidad aprovisionada

Caracteristica	PAY_PER_REQUEST	Aprovisionada
Cobro	Por operacion (RCU/WCU consumidas)	Por hora, aunque no haya trafico
Escala	Automatica	Manual o Auto Scaling
Tabla en reposo	$0.00	Sigue cobrando
Ideal para	Trafico impredecible, proyectos escolares	Cargas estables y predecibles

Captura setup-A - Tabla eliminada y recreando con SK compuesta + GSI



Captura setup-B - JSON completo: GSI_fecha, BillingMode=PAY_PER_REQUEST, ARN



Captura setup-C - Tabla ACTIVE + GSI_fecha ACTIVE + diseno de clave documentado

```
Tabla enviada a crear. Esperando estado ACTIVE...

--- Estado de la tabla ---
-------------------------------------------------------------------------------
|                                DescribeTable                                |
+--------+------------------+-------------------------+--------------+---------+
| Estado |      Modo        |         Nombre          |     PK       |   SK    |
+--------+------------------+-------------------------+--------------+---------+
|  ACTIVE|  PAY_PER_REQUEST |  movimientos_inventario  |  producto_id |  fecha  |
+--------+------------------+-------------------------+--------------+---------+

--- Índices secundarios globales ---
-----------------------------------------------------------------
|                        DescribeTable                          |
+--------+------------+------------+--------------+---------+
| Estado |  Nombre    |    PK      | Proyeccion   |   SK    |
+--------+------------+------------+--------------+---------+
|  ACTIVE|  GSI_fecha |  fecha_dia |  ALL         |  fecha  |
+--------+------------+------------+--------------+---------+

✓  Tabla 'movimientos_inventario' lista.

DISEÑO DE CLAVE:
  PK  : producto_id              (String)
  SK  : fecha                    (String — 'ISO#movimiento_id')
        Ej: '2026-10-14T09:15:00Z#MOV-8810'
        El sufijo evita que dos movimientos del mismo producto
        en el mismo segundo se sobrescriban (son ítems distintos).

  GSI_fecha:
  PK  : fecha_dia  (YYYY-MM-DD)   — permite corte diario sin Scan
  SK  : fecha      (ISO#movimiento_id) — permite BETWEEN para rangos
  Q3 (rango de fechas): iterar fecha_dia = '2026-10-14' con GSI_fecha
```

4. Criterio 1 - Insertar elementos (put_item desde codigo Python)

Codigo: `operaciones_dynamodb.py` ejecutado en la instancia EC2 `i-0ab6dbe715f440427` con el Instance Profile `ip-rol-ec2-s3-los-rojos`.

Las credenciales **no estan en el codigo ni en variables de entorno**: boto3 las obtiene automaticamente del **Instance Metadata Service (IMDS)** de la EC2. El IMDS entrega credenciales temporales del rol `rol-ec2-s3-los-rojos` (politica `pol-inline-dynamodb-los-rojos`). La funcion `sts.get_caller_identity()` al inicio del script confirma que el ARN activo contiene `assumed-role`, no un usuario con llaves permanentes.

Fragmento clave del codigo (operaciones_dynamodb.py)

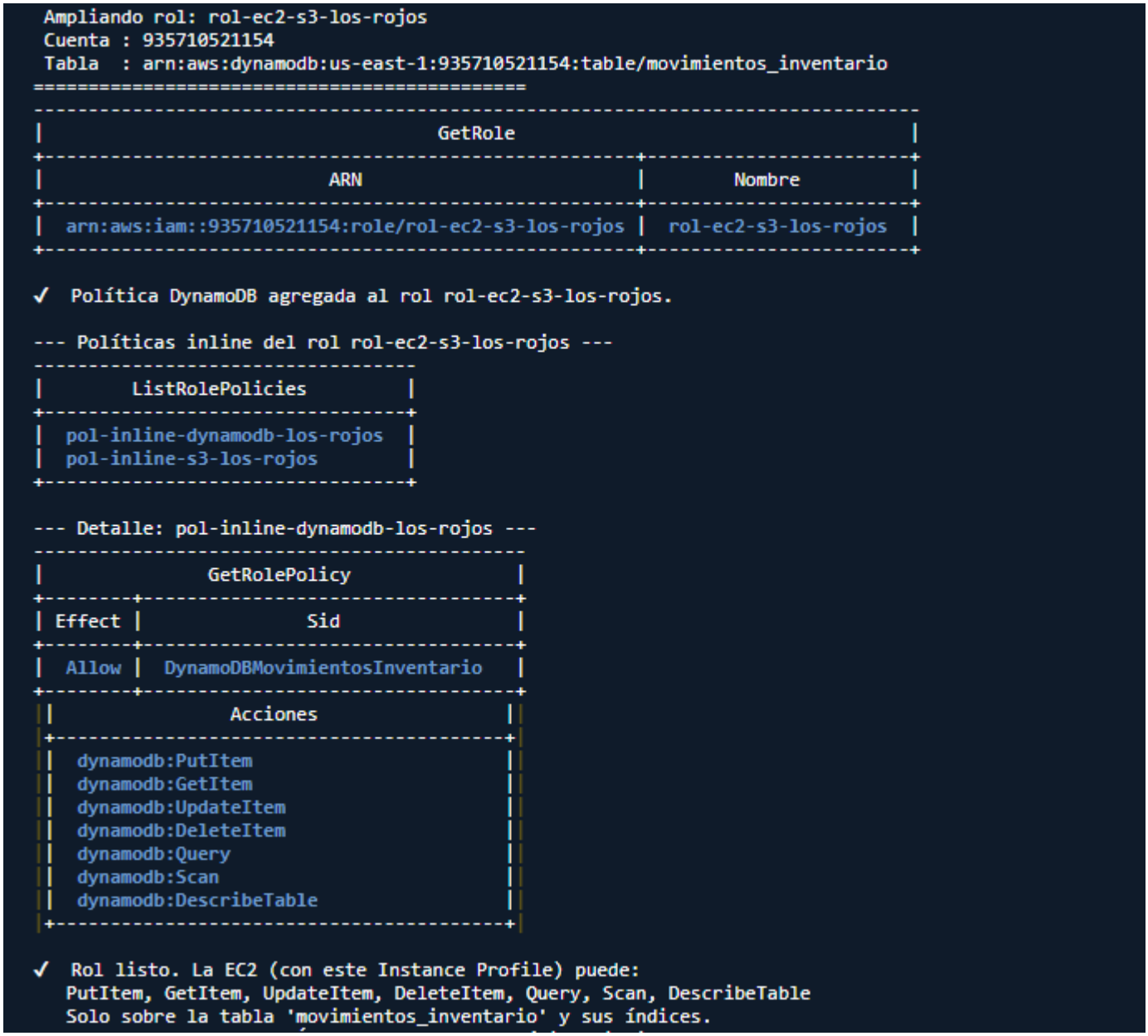
```
import boto3
dynamodb = boto3.resource('dynamodb', region_name='us-east-1')  # sin llaves
table    = dynamodb.Table('movimientos_inventario')
# boto3 resuelve credenciales: env -> ~/.aws -> IMDS (rol EC2) <-- este caso

table.put_item(Item={
    'producto_id'   : 'PROD-0042',
    'fecha'         : '2026-10-14T09:15:00Z#MOV-8810',  # SK compuesta
    'movimiento_id' : 'MOV-8810',
    'fecha_dia'     : '2026-10-14',                      # atributo para GSI
    ...                                                   # resto de atributos
})
```

Por que SK compuesta 'ISO-8601#movimiento_id'

Con `fecha` como unica SK, dos movimientos del mismo producto en el mismo segundo producen la misma clave (producto_id, fecha): el segundo `put_item` sobrescribe al primero silenciosamente. Añadir `#MOV-XXXX` garantiza unicidad sin perder el orden cronologico (el prefijo ISO-8601 sigue siendo lexicograficamente ordenable).

Captura rol - pol-inline-dynamodb-los-rojos + pol-inline-s3-los-rojos adjuntas al rol EC2



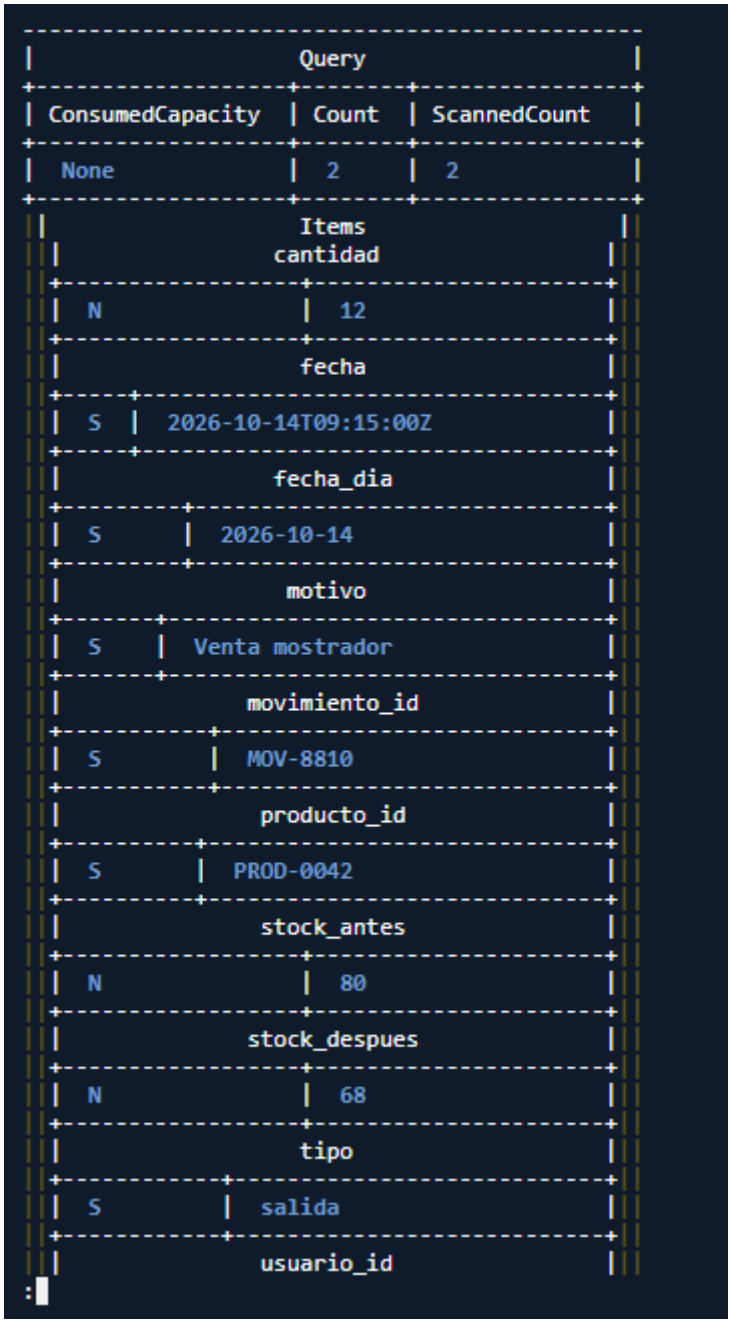
Elementos insertados

movimiento_id	producto_id	SK (fecha)	tipo	cantidad	stock despues
MOV-8810	PROD-0042	2026-10-14T09:15:00Z#MOV-8810	salida	12	68
MOV-8801	PROD-0042	2026-10-13T14:30:00Z#MOV-8801	entrada	50	80
MOV-8815	PROD-0017	2026-10-14T11:00:00Z#MOV-8815	ajuste	-3	12

Captura C1-A - 3 elementos insertados - SK compuesta visible en la salida



Captura C1-B - Verificacion con Query de PROD-0042 (Count=2)


Query
ConsumedCapacity
Count
ScannedCount
None
2
2
Items
cantidad
N
12
fecha
S
2026-10-14T09:15:00Z
fecha_dia
S
2026-10-14
motivo
S
Venta mostrador
movimiento_id
S
MOV-8810
producto_id
S
PROD-0042
stock_antes
N
80
stock_despues
N
68
tipo
S
salida
usuario_id

5. Criterio 2 - Consultar (query)

Se ejecuto `evidencia-p07-c2-consultar.sh`. Ambas consultas usan la clave de particion, por lo que DynamoDB localiza la particion directamente: **costo minimo, sin escanear la tabla completa**.

Q1 - Todos los movimientos de PROD-0042, mas recientes primero

`KeyConditionExpression: producto_id = :pid | --no-scan-index-forward`

Captura C2-A - Q1: 2 items de PROD-0042 ordenados DESC

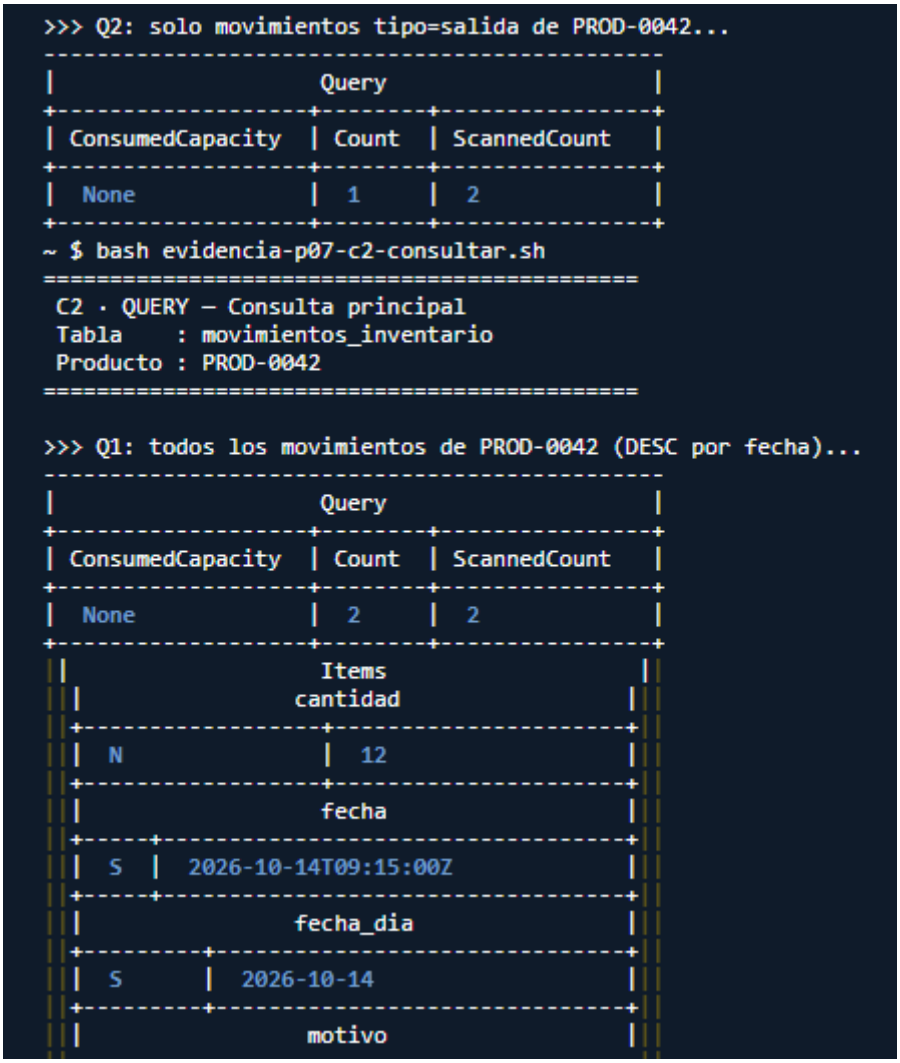
```
>>> Q2: solo movimientos tipo=salida de PROD-0042...
------------------------------------------------
|                   Query                      |
+------------------+--------+----------------+
| ConsumedCapacity | Count  | ScannedCount   |
+------------------+--------+----------------+
|  None            |  1     |  2             |
+------------------+--------+----------------+
~ $ bash evidencia-p07-c2-consultar.sh
================================================
 C2 · QUERY — Consulta principal
 Tabla    : movimientos_inventario
 Producto : PROD-0042
================================================

>>> Q1: todos los movimientos de PROD-0042 (DESC por fecha)...
------------------------------------------------
|                   Query                      |
+------------------+--------+----------------+
| ConsumedCapacity | Count  | ScannedCount   |
+------------------+--------+----------------+
|  None            |  2     |  2             |
+------------------+--------+----------------+
||                  Items                    ||
||               cantidad                    ||
||+-----------------+-----------------------+||
|||  N              |  12                   |||
||+-----------------+-----------------------+||
||                  fecha                    ||
||+-----+-----------------------------------+||
|||  S  |  2026-10-14T09:15:00Z             |||
||+-----+-----------------------------------+||
||               fecha_dia                   ||
||+---------+---------------------------------+||
|||  S      |  2026-10-14                     |||
||+---------+---------------------------------+||
||                 motivo                    ||
||+-------+-----------------------------------+||
|||  S    |  Venta mostrador                  |||
||+-------+-----------------------------------+||
||              movimiento_id                  ||
||+-----------+-----------------------------+||
|||  S        |  MOV-8810                   |||
||+-----------+-----------------------------+||
```

Q2 - Solo movimientos tipo=salida de PROD-0042

`KeyConditionExpression: producto_id = :pid` + `FilterExpression: tipo = salida`

Captura C2-B - Q2: Count=1, ScannedCount=2 - solo MOV-8810



Nota sobre ScannedCount: Q2 devuelve Count=1 pero ScannedCount=2. DynamoDB leyo los 2 items de la particion y luego aplico el filtro. En una tabla con miles de movimientos por producto esto escala - la solucion correcta es un GSI adicional con tipo como clave.

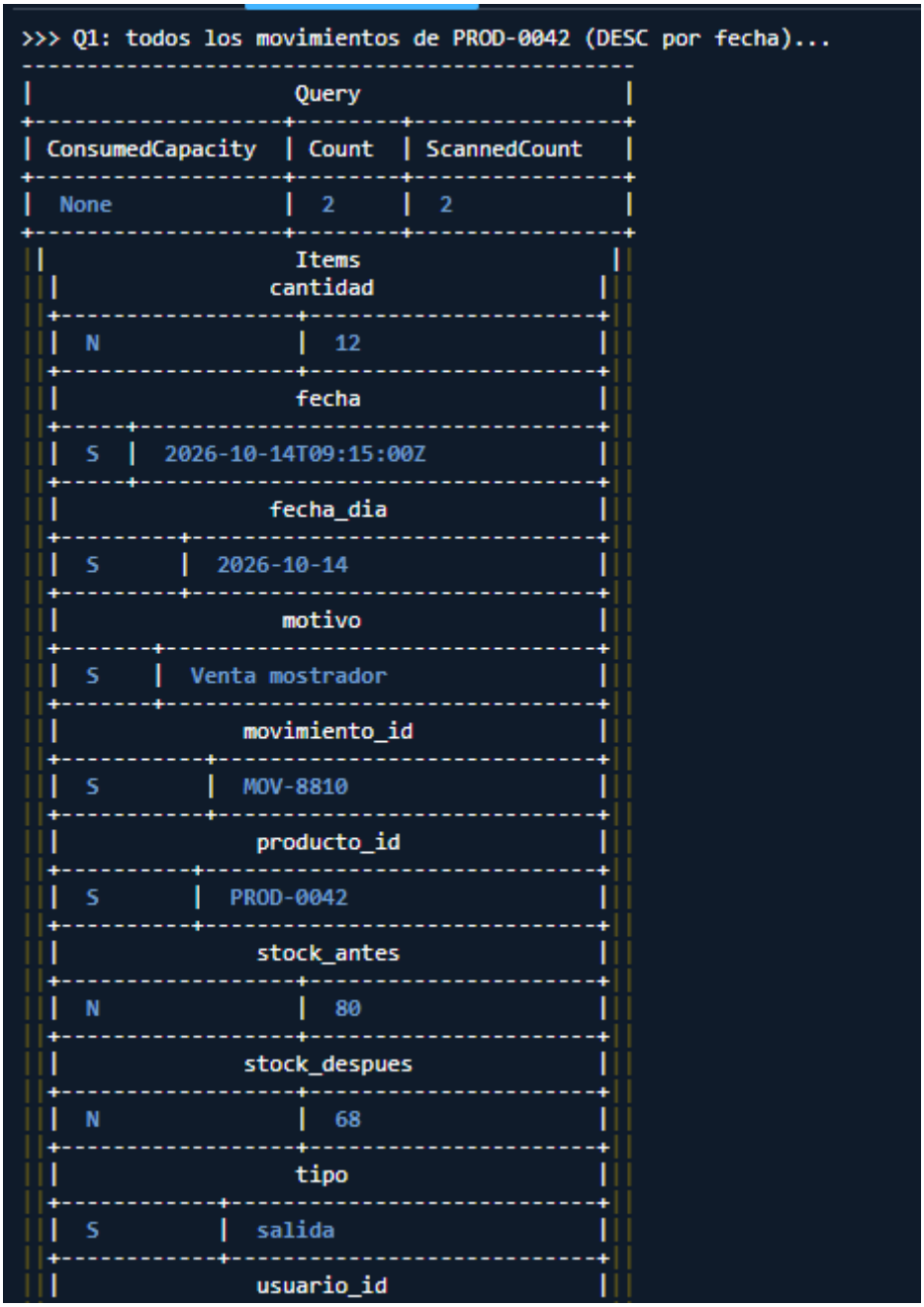
6. Criterio 3 - Actualizar elemento (update-item)

Se ejecuto `evidencia-p07-c3-actualizar.sh`. Se actualizaron `motivo` y `stock_despues` de MOV-8810. El script muestra el estado antes y despues con `--return-values ALL_NEW`.

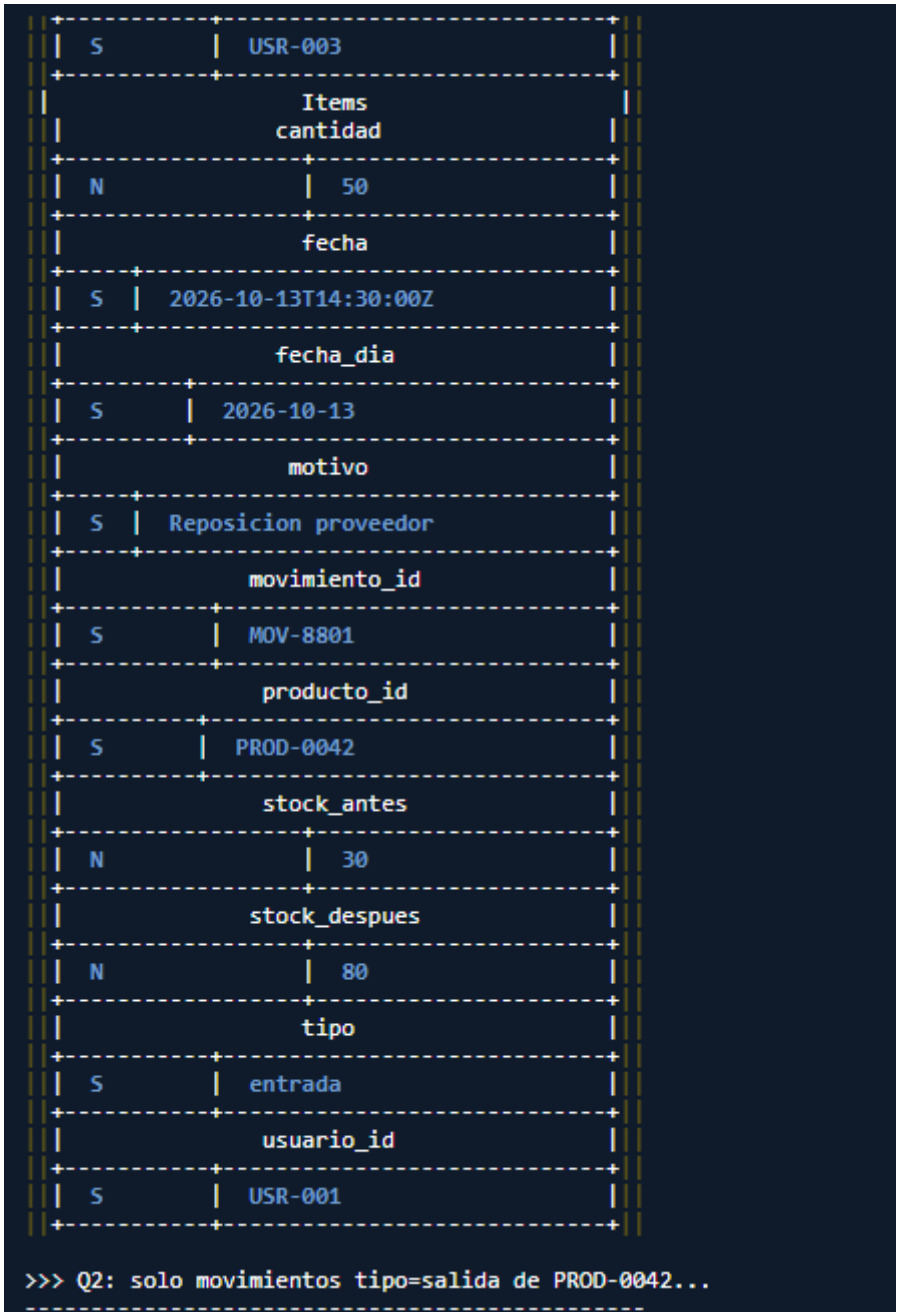
Cambios aplicados

Atributo	Antes	Despues
motivo	Venta mostrador	Venta mostrador - corregida
stock_despues	68	67

Captura C3-A - Estado ANTES (motivo: Venta mostrador, stock_despues: 68)



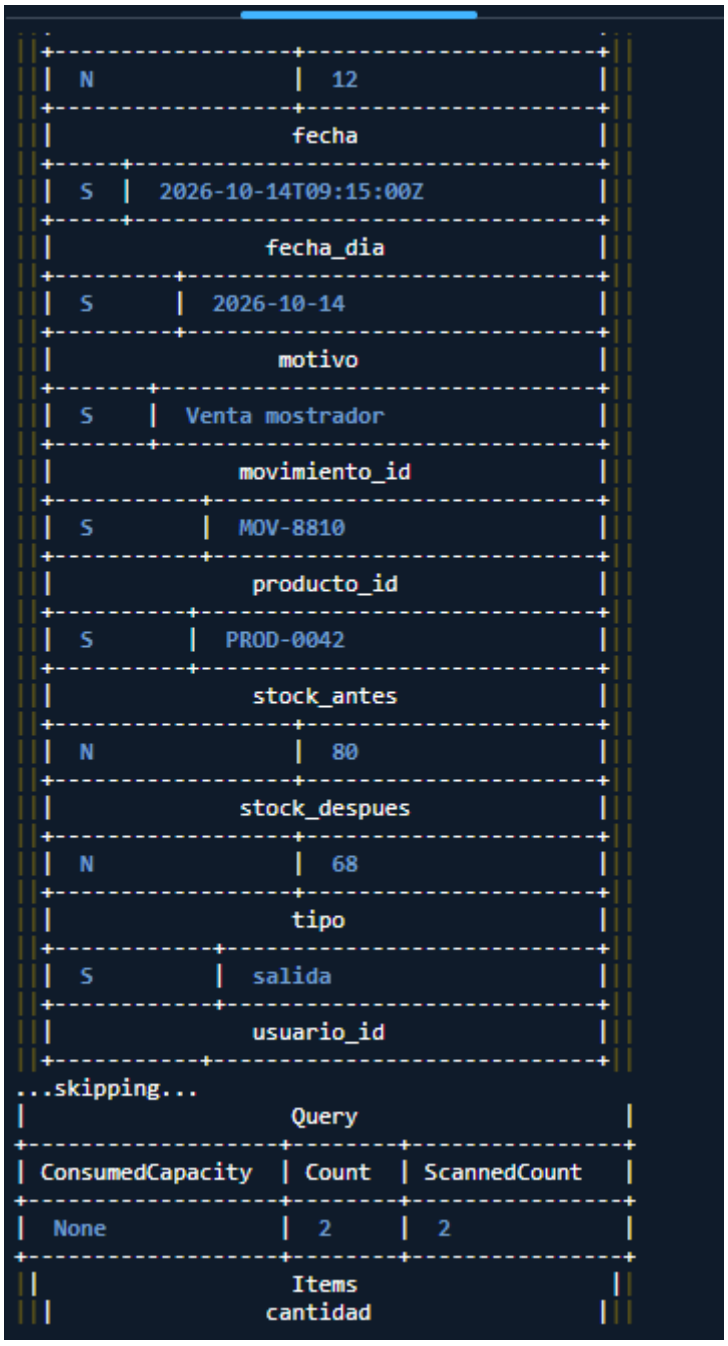
Captura C3-B - Estado DESPUES: motivo corregido, stock_despues=67



7. Criterio 4 - Eliminar elemento (delete_item)

Se ejecuto `evidencia-p07-c4-eliminar.sh`. El script: (1) verifica que MOV-8815 existe con `get-item`, (2) lo elimina con `--return-values ALL_OLD` para devolver los atributos, y (3) confirma la eliminacion con un segundo `get-item` que debe devolver respuesta vacia.

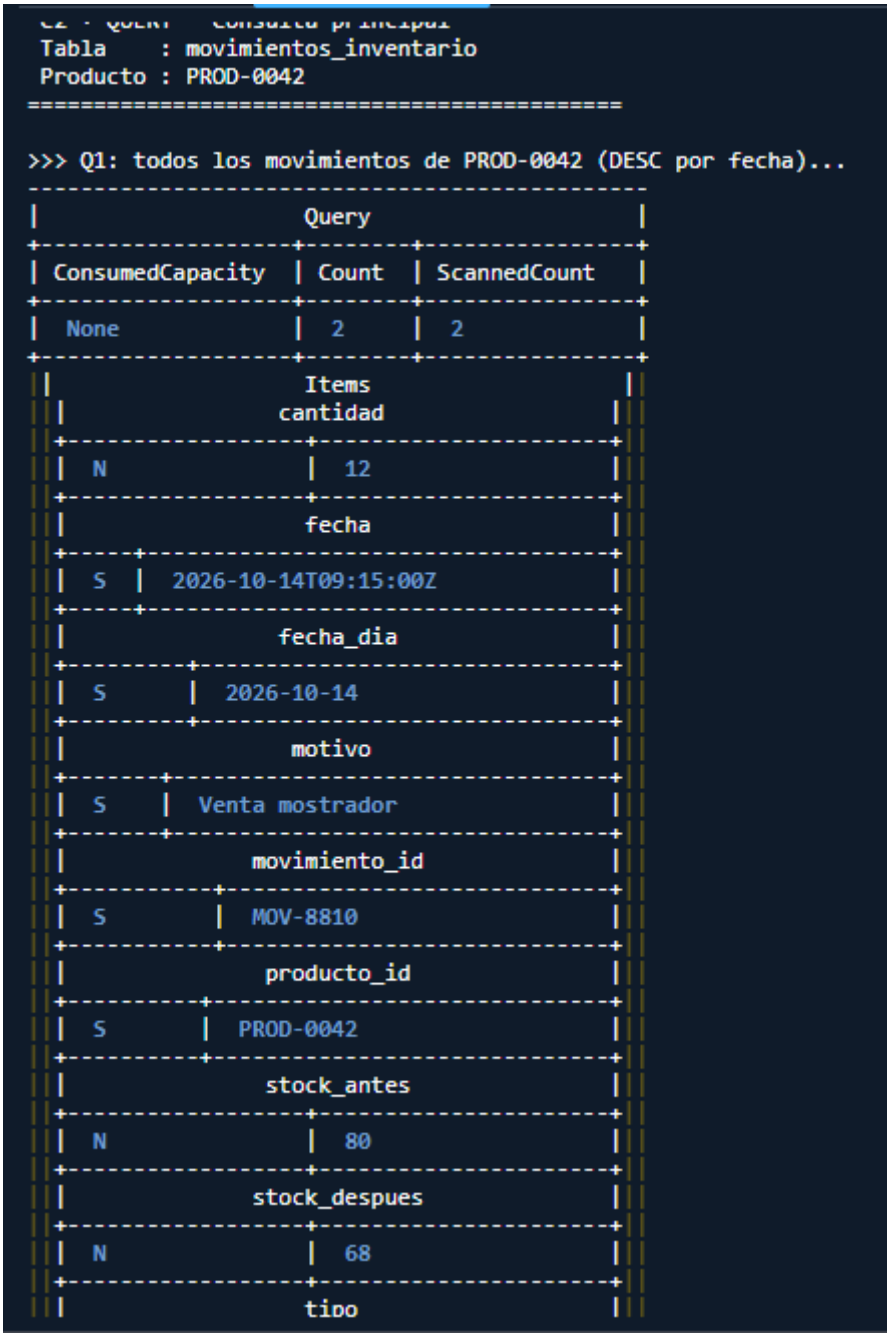
Captura C4-A - GetItem ANTES: MOV-8815 existe (Merma por caducidad)



Captura C4-B - DeleteItem: atributos devueltos (ALL_OLD) - confirma que el item existia



Captura C4-C - GetItem DESPUES: respuesta vacia - item eliminado confirmado



Por que ALL_OLD importa: ReturnValues=ALL_OLD es la unica forma de confirmar que DynamoDB realmente borro el item correcto. Sin el, el DeleteItem devuelve solo HTTP 200 sin datos — no hay forma de distinguir 'borre el item' de 'el item no existia'.

8. Consulta ineficiente - Scan sin clave de particion

Se ejecuto `evidencia-p07-c5-scan-ineficiente.sh`. DynamoDB recorre **todos los elementos de la tabla** sin importar cuantos coincidan con el filtro.

Resultados del Scan (tabla real, 5 items)

Metrica	Valor	Interpretacion
Items devueltos	1	Solo MOV-8810 es tipo=salida (despues del DELETE de MOV-8815)
Items escaneados	2	Recorrio TODOS los items de la tabla sin importar el filtro
Capacidad usada	2.0 RCUs	Eventually consistent: 1 RCU por cada 4 KB leidos (0.5 RCU x 4 KB = 2 RCU)

Calculo de costo a escala — supuesto declarado

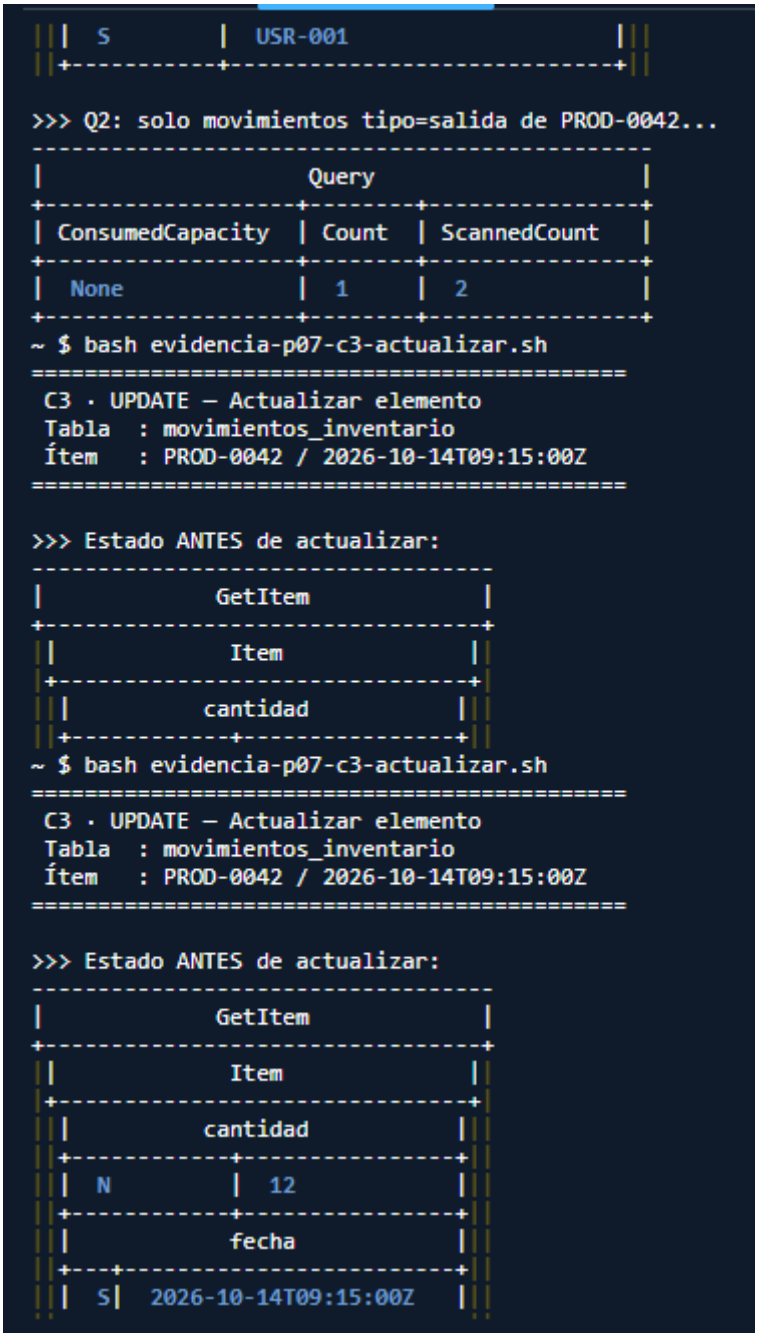
Supuesto: item promedio = 200 B (8 atributos × ~25 B cada uno, sin atributos de texto largo).

Precio on-demand us-east-1: $0.25 USD por millon de RCUs (eventually consistent: 0.5 RCU por cada 4 KB leidos).

Operacion	Items / datos leidos	RCUs consumidas	Costo/ejecucion	Si corre c/min (30 dias)
Scan - 1M items x 200B	200 MB totales	ceil(200MB/4KB) x 0.5 = 24,415 RCUs	$0.0061 USD	~$263 USD/mes
Query - 1 producto, 10 movs x 200B	2,000 B (~2 KB)	ceil(2KB/4KB) x 0.5 = 0.5 RCUs	$0.000000125 USD	~$0.005 USD/mes

La Query es **~49 000 veces mas barata** que el Scan en este escenario. A mayor tamano de item (ej. 1 KB), el Scan seria 5 veces mas caro que el ejemplo; a menor tamano (50 B), seria mas barato pero seguiria siendo ~49 000x mas caro que la Query equivalente.

Captura C5-A - Scan ejecutando: Count=1, ScannedCount=2, CapacityUnits=2.0



Captura C5-B - Resumen de consumo con calculo de costo proyectado



Conclusion: El Scan es aceptable en desarrollo con pocos elementos. En produccion con millones de registros es **prohibitivamente caro**. El diseno con PK=producto_id (Q1, Q2) y GSI_fecha (Q3) garantiza que ningun patron de acceso necesita Scan.

9. Comparacion: modelo relacional vs. no relacional

Criterio	PostgreSQL (P06)	DynamoDB (P07)
Diseno	Modelo por entidades (ERD, normalizacion)	Modelo por consultas (access pattern, PK/SK)
Consultas	SQL flexible: cualquier columna, JOINs	Eficientes solo si usan la clave de particion
Escalabilidad	Vertical	Horizontal automatica
Costo en reposo	~$0.023/hora aunque no haya trafico	$0.00 con PAY_PER_REQUEST
Relaciones	Claves foraneas, integridad referencial	Sin JOINs, desnormalizacion necesaria
Mas facil	Consultas ad-hoc con SQL	Insertar y consultar por clave
Mas dificil	Configurar red privada y permisos	Disenar la clave correcta y el GSI sin patron previo

Cual conviene al proyecto Inventario Nube?

Respuesta: ambas, para tablas distintas.

- **PostgreSQL** para el catalogo (productos, categorias, usuarios): requiere JOINs y consultas por multiples columnas con esquema estable.
- **DynamoDB** para movimientos de inventario: crece sin limite, siempre se consulta por producto_id y fecha, y no requiere JOINs. Exactamente el patron que DynamoDB optimiza.

Criterio de seleccion para el documento final: **el motor se elige segun el patron de acceso, no por familiaridad.**

10. Lista de cotejo

Criterio	Pts	Evidencia	Cumple
El patron de acceso esta definido antes del diseno	20	Seccion 2: Q1, Q2, Q3 antes de crear la tabla	Si
La clave de particion esta justificada y resuelve las consultas	20	Seccion 2: producto_id resuelve Q1 y Q2 en O(1)	Si
Las cuatro operaciones funcionan con evidencia	25	Secciones 4-7: INSERT, QUERY, UPDATE, DELETE	Si
El codigo usa rol de servicio sin credenciales	15	Sec. 4: Python boto3 en EC2 con ip-rol-ec2-s3-los-rojos; ARN assumed-role confirma rol	Si
La comparacion entre modelos usa criterio explicito	20	Seccion 9: patron de acceso como criterio	Si
TOTAL	100		

11. Conclusiones

La Practica 07 completo el ciclo de base de datos iniciado en la practica anterior. La tabla `movimientos_inventario` quedo operativa en DynamoDB con las cuatro operaciones implementadas, el patron de acceso documentado antes del diseno, y la clave de particion justificada con criterio tecnico.

La demostracion del Scan ineficiente fue el aprendizaje mas concreto: asumiendo items de 200 B, el mismo patron a un millon de elementos consume ~24 415 RCUs por ejecucion (~$0.006 USD). A una ejecucion por minuto durante 30 dias el gasto supera **$263 USD/mes** — solo por esta operacion. La Query equivalente cuesta $0.005 USD/mes en el mismo escenario (49 000x mas barata).

La comparacion entre PostgreSQL (P06) y DynamoDB (P07) establece el criterio de seleccion que se usara en el documento tecnico final: **el motor se elige segun el patron de acceso.** El catalogo de productos queda en PostgreSQL; los movimientos son candidatos naturales para DynamoDB.

9. Versionado del codigo

Todos los scripts y el codigo Python estan versionados en el repositorio del equipo:

Repositorio: `https://github.com/Angektam/Computo-en-la-nube--Servidor`

Rama: `main`

Archivos versionados en esta practica:

-
-
-
-
-
-
-
-
-

Relacion con el entregable E2

- La justificacion del motor de base de datos esta documentada con evidencia empirica.
- El codigo de acceso a datos esta versionado en el repositorio del equipo.
- El analisis de patron de acceso sustenta el diseno del modelo de datos del apartado 9 del documento final.